API-ДОКУМЕНТАЦИЯ

Приложение для изучения иностранных языков

REST API · Версия 1.0 · Учебный проект

Назначение документа

Документ описывает контракт взаимодействия мобильного/веб-клиента с сервером учебного приложения: настройки обучения, тестирование уровня, уроки, прогресс, жизни и подписки. Примеры являются проектным предложением для учебного задания, а не описанием уже работающего сервера.

Базовый URL: https://api.language-app.example/api/v1

Формат данных: JSON · Кодировка: UTF-8 · Протокол: HTTPS

1. Общие правила API

API предоставляет REST-интерфейс для клиентского приложения. Все пути ниже указаны относительно базового URL https://api.language-app.example/api/v1. Домен example зарезервирован для примеров; перед реальной интеграцией его необходимо заменить на адрес сервера проекта.

Заголовки запросов

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer <access_token>
```

Для публичных методов заголовок Authorization не требуется. Для защищённых методов клиент передаёт действительный access token. Токены, пароли и платёжные данные нельзя записывать в логи.

Формат ошибки

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "[redacted]",
    "details": [
      { "field": "daily_goal_minutes", "issue": "[redacted] 5, 10 [redacted] 15" }
    ],
    "request_id": "req_01HXYZ"
  }
}
```

Основные HTTP-коды

Код	Значение	Когда используется
200	OK	Запрос успешно выполнен
201	Created	Ресурс создан
400	Bad Request	Неверный формат или параметры запроса
401	Unauthorized	Токен отсутствует или недействителен
403	Forbidden	Недостаточно прав
404	Not Found	Ресурс не найден
409	Conflict	Конфликт состояния, например повторная операция
422	Unprocessable Entity	Данные синтаксически верны, но не проходят бизнес-проверку
429	Too Many Requests	Превышен лимит запросов
500	Internal Server Error	Непредвиденная ошибка сервера

2. Методы API

2.1. Создание пользователя

Создание учётной записи

POST /users

Назначение: Создаёт пользователя и возвращает идентификатор. Для учебного примера пароль не передаётся в открытом виде в ответе.

Доступ: Публичный; в production требуется проверка email и безопасное хранение пароля.

Связь с User Story: Старт: пользователь начинает обучение.

Пример запроса

```
{
  "email": "student@example.com",
  "password": "StrongPassword123!"
}
```

Пример успешного ответа

```
{
  "user": {
    "id": "usr_123",
    "email": "student@example.com",
    "created_at": "2026-10-20T18:00:00Z"
  }
}
```

Ошибки: 400 — неверный формат email или пароля; 409 — email уже зарегистрирован.

Сохранение учебных настроек

PUT /users/me/preferences

Назначение: Сохраняет язык обучения и ежедневную цель занятий.

Доступ: Авторизованный пользователь; изменяет только собственные настройки.

Связь с User Story: Старт US-1 и US-3: выбрать язык и длительность занятий.

Пример запроса

```
{
  "learning_language": "es",
  "daily_goal_minutes": 10
}
```

Пример успешного ответа

```
{
  "learning_language": "es",
  "daily_goal_minutes": 10,
  "updated_at": "2026-10-20T18:05:00Z"
}
```

Ошибки: 400 — неподдерживаемый язык или значение цели вне 5, 10, 15; 401 — нет действительного токена.

Получение учебных настроек

GET /users/me/preferences

Назначение: Возвращает сохранённые настройки пользователя.

Доступ: Авторизованный пользователь.

Связь с User Story: Старт: показать ранее выбранный язык и цель при повторном входе.

Пример успешного ответа

```
{
  "learning_language": "es",
  "daily_goal_minutes": 10,
  "placement_level": "A1"
}
```

Ошибки: 401 — нет действительного токена; 404 — настройки ещё не созданы.

Начало теста уровня

POST /placement-tests

Назначение: Создаёт попытку тестирования и возвращает первый вопрос.

Доступ: Авторизованный пользователь.

Связь с User Story: Старт US-2: определить текущий уровень и не начинать обучение с неподходящего материала.

Пример запроса

```
{
  "learning_language": "es"
}
```

Пример успешного ответа

```
{
  "test_id": "test_456",
  "question": {
    "id": "q_001",
    "type": "single_choice",
    "text": "[redacted] «hola».",
    "options": ["[redacted]", "[redacted]", "[redacted]"]
  }
}
```

Ошибки: 400 — язык не выбран; 401 — пользователь не авторизован.

Отправка ответа теста уровня

POST /placement-tests/{testId}/answers

Назначение: Проверяет ответ, фиксирует результат и возвращает следующий вопрос либо итоговый уровень.

Доступ: Авторизованный пользователь — владелец попытки тестирования.

Связь с User Story: Старт US-2; пропуск теста для новичка может быть добавлен отдельным сценарием в релизе 2.

Пример запроса

```
{
  "question_id": "q_001",
  "answer": "[redacted]"
}
```

Пример успешного ответа

```
{
  "is_correct": true,
  "next_question": {
    "id": "q_002",
    "type": "single_choice",
    "text": "[redacted] «gracias».",
    "options": ["[redacted]", "[redacted]", "[redacted]"]
  },
  "completed": false
}
```

Ошибки: 400 — некорректный ответ; 404 — тест или вопрос не найден; 409 — тест уже завершён.

Получение списка уроков

GET /lessons

Назначение: Возвращает доступные уроки выбранного языка с признаком доступности.

Доступ: Авторизованный пользователь.

Связь с User Story: Урок US-1 и мотивация US-1: пользователь выбирает следующий урок через путь навыков.

Пример успешного ответа


```
{
  "items": [
    {
      "id": "lesson_001",
      "title": "[redacted]",
      "skill_id": "skill_001",
      "status": "available",
      "order": 1
    },
    {
      "id": "lesson_002",
      "title": "[redacted]",
      "skill_id": "skill_001",
      "status": "locked",
      "order": 2
    }
  ]
}
```


Ошибки: 401 — нет действительного токена.

Получение содержимого урока

GET /lessons/{lessonId}

Назначение: Возвращает задания урока и данные, необходимые для отображения упражнения.

Доступ: Авторизованный пользователь; доступность урока проверяется сервером.

Связь с User Story: Урок US-1: собирать фразы из слов-кубиков.

Пример успешного ответа


```
{
  "id": "lesson_001",
  "title": "[redacted]",
  "progress": { "completed_tasks": 0, "total_tasks": 5 },
  "tasks": [
    {
      "id": "task_001",
      "type": "word_order",
      "prompt": "[redacted] «[redacted], [redacted]!»",
      "tokens": ["¡Hola,", "Anna!"]
    }
  ]
}
```


Ошибки: 401 — нет действительного токена; 403 — урок заблокирован; 404 — урок не найден.

Отправка ответа в уроке

POST /lessons/{lessonId}/answers

Назначение: Проверяет ответ пользователя, обновляет прогресс и при необходимости уменьшает число жизней.

Доступ: Авторизованный пользователь; сервер проверяет принадлежность попытки пользователю.

Связь с User Story: Урок US-2 и US-3: обратная связь за правильный ответ и показ правильного ответа при ошибке; сложности US-1: потеря жизни за ошибку.

Пример запроса

```
{
  "task_id": "task_001",
  "answer": ["¡Hola,", "Anna!"]
}
```

Пример успешного ответа

```
{
  "is_correct": true,
  "feedback": {
    "message": "■■■■■!",
    "show_success_animation": true,
    "play_success_sound": true
  },
  "progress": {
    "completed_tasks": 1,
    "total_tasks": 5,
    "percent": 20
  },
  "lives_remaining": 5
}
```

Ошибки: 400 — отсутствует ответ; 401 — нет токена; 404 — урок или задание не найдено; 409 — задание уже засчитано.

Сохранение прогресса урока

PUT /lessons/{lessonId}/progress

Назначение: Сохраняет позицию пользователя в уроке, чтобы он мог продолжить позже.

Доступ: Авторизованный пользователь.

Связь с User Story: Урок US-6 релиза 2: выйти из урока с сохранением прогресса.

Пример запроса

```
{
  "last_task_id": "task_003",
  "completed_tasks": 3
}
```

Пример успешного ответа

```
{
  "lesson_id": "lesson_001",
  "last_task_id": "task_003",
  "completed_tasks": 3,
  "saved": true
}
```

Ошибки: 400 — некорректный прогресс; 401 — нет токена; 404 — урок не найден.

Получение прогресса и streak

GET /progress

Назначение: Возвращает общую статистику обучения, текущую серию занятий и дерево навыков.

Доступ: Авторизованный пользователь.

Связь с User Story: Мотивация US-1 и US-2: просматривать дерево навыков и серию занятий.

Пример успешного ответа

```
{
  "total_xp": 120,
  "current_streak_days": 4,
  "longest_streak_days": 7,
  "skills": [
    {
      "id": "skill_001",
      "name": "[redacted]",
      "progress_percent": 60,
      "status": "in_progress"
    }
  ]
}
```

Ошибки: 401 — нет действительного токена.

Настройка напоминаний

PUT /users/me/reminders

Назначение: Сохраняет настройки вечерних напоминаний о занятиях.

Доступ: Авторизованный пользователь.

Связь с User Story: Мотивация US-3: получать напоминание, чтобы продолжать streak.

Пример запроса

```
{
  "enabled": true,
  "local_time": "20:00",
  "timezone": "Europe/Amsterdam"
}
```

Пример успешного ответа

```
{
  "enabled": true,
  "local_time": "20:00",
  "timezone": "Europe/Amsterdam",
  "updated": true
}
```

Ошибки: 400 — неверное время или часовой пояс; 401 — нет токена.

Получение количества жизней

GET /lives

Назначение: Возвращает текущее количество жизней и время их восстановления, если оно применимо.

Доступ: Авторизованный пользователь.

Связь с User Story: Сложности и деньги US-1; таймер восстановления жизней относится к релизу 2.

Пример успешного ответа

```
{
  "current": 4,
  "maximum": 5,
  "next_recovery_at": "2026-10-20T18:30:00Z"
}
```

Ошибки: 401 — нет действительного токена.

Получение тарифов подписки

GET /subscriptions/plans

Назначение: Возвращает доступные тарифы и список их преимуществ.

Доступ: Публичный или авторизованный пользователь.

Связь с User Story: Сложности и деньги US-2: просмотреть преимущества Super; покупка подписки и семейный тариф реализуются в последующих релизах.

Пример успешного ответа

```
{
  "plans": [
    {
      "id": "super_monthly",
      "name": "Super",
      "billing_period": "month",
      "features": ["[redacted]", "[redacted]"]
    },
    {
      "id": "family_monthly",
      "name": "Super Family",
      "billing_period": "month",
      "features": ["[redacted] Super [redacted]"]
    }
  ]
}
```

Ошибки: 500 — ошибка сервера при загрузке тарифов.

3. Дополнительные методы будущих релизов

Следующие методы расширяют функциональность после MVP. Их можно реализовать после подтверждения основных пользовательских сценариев.

Получение произношения слова

GET /pronunciation/{wordId}

Назначение: Возвращает URL аудиозаписи произношения и языковую метку.

Доступ: Авторизованный пользователь.

Связь с User Story: Урок US-5 релиза 2: прослушать произношение слова.

Пример успешного ответа

```
{
  "word_id": "word_123",
  "language": "es",
  "audio_url": "https://cdn.example/audio/word_123.mp3"
}
```

Ошибки: 404 — слово или аудио не найдено; 401 — нет токена.

Получение таблицы лидеров

GET /leaderboard

Назначение: Возвращает рейтинг пользователей за выбранный период.

Доступ: Авторизованный пользователь.

Связь с User Story: Мотивация US-4 релиза 2.

Пример успешного ответа

```
{
  "period": "weekly",
  "items": [
    { "rank": 1, "display_name": "Luna", "xp": 450 },
    { "rank": 2, "display_name": "Alex", "xp": 420 }
  ]
}
```

Ошибки: 400 — неподдерживаемый период; 401 — нет токена.

Создание сессии оформления подписки

POST /subscriptions/checkout-sessions

Назначение: Создаёт сессию перехода к платёжному провайдеру. Сам платёж обрабатывается провайдером, а не через передачу данных банковской карты этому API.

Доступ: Авторизованный пользователь.

Связь с User Story: Сложности и деньги US-4 релиза 2; семейный тариф — релиз 3.

Пример запроса

```
{
  "plan_id": "super_monthly",
  "success_url": "https://app.example/subscription/success",
  "cancel_url": "https://app.example/subscription/cancel"
}
```

Пример успешного ответа

```
{
  "checkout_url": "https://payments.example/checkout/session_123",
  "session_id": "checkout_123"
}
```

Ошибки: 400 — неизвестный тариф или некорректные URL; 401 — нет токена; 503 — платёжный провайдер недоступен.

4. Правила валидации и безопасности

• Все защищённые методы должны проверять access token и идентификатор владельца ресурса.

• Сервер валидирует значения, даже если клиент уже проверил их на экране.

• Пароли хранятся только в виде стойкого криптографического хеша; в ответах API пароли никогда не возвращаются.

• Для оплаты используется платёжный провайдер. Нельзя передавать или хранить полный номер карты и CVV в собственном API.

• Для отправки ответов и создания платежных сессий рекомендуется поддержать идемпотентность, чтобы повтор запроса при сетевой ошибке не создавал дубли.

• Временные метки передаются в ISO 8601 / UTC. Время напоминаний задаётся локальным временем и IANA-часовым поясом пользователя.

5. Соответствие пользовательским историям

Этап / User Story	Методы API	Релиз
Старт: язык и ежедневная цель	PUT/GET /users/me/preferences	MVP
Старт: тест уровня	POST /placement-tests; POST /placement-tests/{testId}/answers	MVP
Урок: задания и проверка ответов	GET /lessons; GET /lessons/{lessonId}; POST /lessons/{lessonId}/answers	MVP
Урок: сохранение прогресса	PUT /lessons/{lessonId}/progress	Релиз 2
Мотивация: навыки и streak	GET /progress	MVP
Мотивация: напоминания	PUT /users/me/reminders	MVP
Жизни и таймер	GET /lives	MVP / Релиз 2
Подписка и семейный план	GET /subscriptions/plans; POST /subscriptions/checkout-sessions	MVP / Релиз 2 / Релиз 3
Произношение и лидерборд	GET /pronunciation/{wordId}; GET /leaderboard	Релиз 2

6. Примечания к реализации

Документация описывает предполагаемый контракт API для учебного проекта. Перед промышленным использованием необходимо согласовать модель данных, способ аутентификации, лимиты запросов, требования к хранению персональных данных, интеграцию с платёжным провайдером и фактическую бизнес-логику начисления XP, streak и жизней.

Термины: endpoint — адрес метода API; request — запрос клиента; response — ответ сервера; MVP — минимально жизнеспособный продукт; streak — серия дней с занятиями.